

URL rewriting and cookies

- At the first request, the server cannot know whether the session cookie will be returned, since it has not yet been received back from the client...
- Thus:
 - Tomcat creates a HttpSession and sends a JSESSIONID
 - If the cookie has not yet been returned, Tomcat cannot be sure that the client accepts it
 - For safety, it adds the JSESSIONID to the generated URLs

```
<body>
<form action="/validation;jsessionid=C9B4833DE38F9FF892A5D527AFCA3517" method = post>
  <label for="uname">User name</label><br>
  <input type="text" id="uname" name="username" value="" required><br><br>
  <label for="pwd">Password</label><br>
  <input type="password" id="pwd" name="password" value="" required><br><br>
  <input type="submit" value="Log in">
</form>
</body>
```



localhost:8080/validation;jsessionid=8BF7E8CA73C6BCA3C4E4AB69440E02B9

URL Rewriting is a mechanism by which the server adds the session ID directly into the URLs

132

URL rewriting and cookies

- Recommended approach:
 - Using session's cookie
 - Disabling URL rewriting
- Why avoid URL Rewriting
 - URLs with ;jsessionid can be:
 - Logged in server logs
 - Saved in bookmarks
 - Accidentally shared (risk of session hijacking)

```
server.servlet.session.tracking-modes=cookie
```

133

True or False?

- In Spring-based web apps, the Front Controller is typically implemented using a single servlet
- A HTTP request must contains at least a header
- In Spring, a bean is always instantiated manually by the developer
- If two CSS selectors have the same specificity, the cascading algorithm examines their origin
- The @Autowired annotation is used to explicitly create a bean in Spring
- XML uses a set of tags predefined by the W3C

134

Spot the error!

Una caratteristica fondamentale dei CSS è il cascading algorithm, che consente agli stili di sovrascriversi o ereditarsi l'un l'altro. Molti modi di includere e combinare le dichiarazioni del foglio di stile possono infatti causare conflitti e contraddizioni. Un conflitto si verifica quando alla stessa proprietà vengono assegnati valori uguali in più dichiarazioni. In questi casi, esiste un sistema di regole che decide quale delle dichiarazioni di stile in conflitto sarà applicata a un elemento. Un foglio di stile può avere tre origini diverse: user-agent, user e author. Tra queste tre origini, il foglio di stile dello user-agent ha la priorità più bassa. Se viene utilizzato un foglio di stile dell'author, che ha una priorità intermedia, il foglio di stile dello user-agent verrà sovrascritto. La parola chiave !importance modifica l'ordine di priorità, tuttavia il foglio di stile dello user rimane a priorità intermedia. Esiste anche una regola di priorità tra i selettori che viene utilizzata quando sussiste un'equivalenza di origine. Secondo tale regola, per ogni selettore viene calcolato un valore che indica il peso del selettore. Questo valore viene definito specificità. La specificità è espressa come valore numerico e quanto più alto è questo valore, tanto più importante è il selettore, che quindi scavalca qualsiasi selettore concorrente con un valore inferiore. Anche i diversi tipi di selettori hanno specificità diverse. Per esempio, un selettore di classe ha una specificità minore di un selettore di elemento. Se due regole CSS hanno la stessa specificità e sono della stessa origine, si usa un'altra regola chiamata ordine di combinazione.

135

A quick recap on concurrency

- Some threads are **concurrent** if they are executed at the same time. These threads may be running at the same time either when multiple processors are available or their execution may be interleaved. Threads may be interleaved in any order
- **Thread safety** is the avoidance of situations in which data is set to either correct or incorrect values, depending on the order in which threads access and modify the data
- **Race conditions** happen when multiple threads read and write the same variable i.e. they access some shared data and they try to modify it at the same time. In such a scenario, threads are “racing” each other to access/change the data

136

Which is the value at the end?

Thread A	Thread B		Int value
			0
read int		←	0
	read int	←	0
increase int			0
	increase int		0
write back int		→	1
	write back int	→	?

137

Beans: scopes and life time

- We came across various beans serving different roles
- But, what about their scope and life time?
- Using Spring we have (at least) the following scopes:
 - Singleton scope
 - Session scope
 - Request scope
- The last 2 scopes are relevant only to web applications, the Singleton scope is more general

138

Singleton scope

- It is the default scope of Spring beans
- Key aspects:
 - Spring creates it during the startup of the application, and it stays alive until Spring is shut down;
 - Just one single instance of the bean, or multiple instances named in different ways are created;
 - The same instance is given (calling `getBean`) or injected into all components that require it;
 - Since there is only one instance of a singleton bean, if the bean's attributes are mutable and accessed concurrently by multiple threads, they must be thread-safe. Thus, making the bean's attributes immutable (using `final`) or another synchronization mechanism is needed.

139

Singleton Scope

- Controller has a singleton scope
- So, we have just one controller shared in the web app receiving all the requests. How is it done?
- Yes, of course we have just one instance of Controller, but its methods are threaded for each request. This implies that local variables to the methods are thread safe, as well as its parameters.
- Pay attention that the Controller attributes are not thread safe!

140

Example

```
@Controller
public class HelloController {

    private String last;

    @GetMapping("/hello")
    public String hello(@RequestParam String name, Model model)
    {
        last = name;
        model.addAttribute("name", last);
        return "hello";
    }
}
```

➡ http://localhost:8080/hello?name=Anna
➡ http://localhost:8080/hello?name=Paolino
➡ http://localhost:8080/hello?name=Paperino



141

Example

```
@Controller
public class HelloController {

    @GetMapping("/hello")
    public String hello(@RequestParam String name, Model model)
    {
        String message = "Ultimo visitatore: " + name;
        model.addAttribute("name", message);
        return "hello";
    }
}
```

➡ http://localhost:8080/hello?name=Anna
 ➡ http://localhost:8080/hello?name=Paolino
 142 ➡ http://localhost:8080/hello?name=Paperino

Ultimo visitatore: Anna
 Ultimo visitatore: Paolino
 Ultimo visitatore: Paperino

Controller and thread-safety

- Controller's attributes:
 - Unmutable attribute + final: THREAD SAFE!
 - Mutable attribute: NOT THREAD SAFE!
 - Solutions:
 - Local variables / parameters
 - Unmutable attribute + final
- Services injected in a Controller:
 - Stateless service: THREAD SAFE!
 - Shared state (mutable) : NOT THREAD SAFE!
 - Solutions:
 - synchronization

143

Session scope

- Key aspects:
 - It is created when a HTTP session is created. It is destroyed when the session expires or it is invalidated;
 - A new instance of the bean is created for each HTTP session (basically for each user), and this instance is shared across multiple requests within the same session;
 - If multiple users are accessing the web app, each of them will get their own instance of the bean;
 - It is typically used for stateful beans that need to maintain state for the duration of a user's session (e.g., shopping carts).

144

Thinking about ...

- A Controller is a singleton:
 - What would happen if we injected a session-scoped bean directly?
 - Would all users share the same instance?
- We know that we have a session for each user...
- What does happen under the hood?

145

Thinking about ...

- At the web app startup, Spring creates the singleton Controller
- A session-scoped bean requires an active HTTP session
.... the real bean cannot be injected
- Spring injects a proxy instead of the real session bean:
 - a proxy is a placeholder injected into the singleton, which dynamically retrieves the correct session-scoped bean instance for each method invocation
- The proxy can exist without an active session
- The controller calls a method on the bean
 - The proxy intercepts the call
 - It retrieves the correct session bean instance
 - Delegates the method call to that instance

146

Again on esempio 15. We can optimize the code a little bit: we can separate the validation part through a service.

OK! Now we have a service managing sensitive data such as username and password. What is the better scope for this service? Singleton?

esercizio 16

147

Request Scope

- Key aspects:
 - It is created at the beginning of each HTTP request and destroyed at the end of it, which means it's short-lived and only available during the request lifecycle;
 - A new instance of it is created for each HTTP request, and this instance is shared across all components that need it within that same request;
 - It is typically used for stateless beans that don't need to retain any state between requests. This scope is often used for request-specific data that doesn't need to persist between multiple requests;
 - It is not prone to multithread related issues as only a thread (the one of the request) can access them

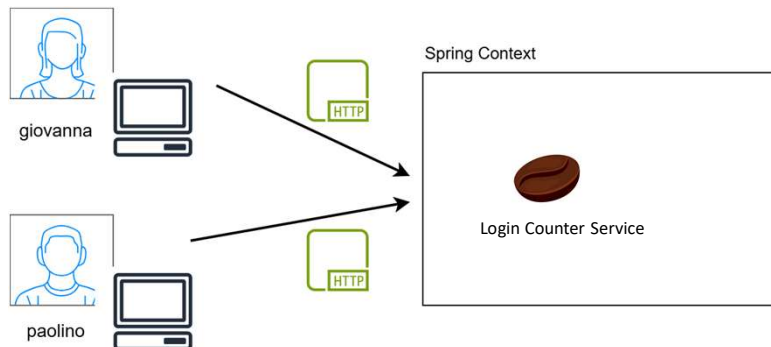
148

Practice@Home

- Modify exercise 16 by annotating the validation service as a Request-scoped bean
- The `@RequestScope` annotation change the bean's scope to request scope. This way, Spring creates a new instance of the class for every HTTP request.

149

Something more on concurrency



- We want implementing a login counter, counting the successful login attempts from all users

esercizio 17

150

Implementing synchronization in Java

- Java provides basic solutions to implement synchronization
- We focus on: synchronized methods

Add the **synchronized** keyword to the method declaration

Two effects:

- It is not possible for two invocations of synchronized methods on the same object to interleave. When one thread is executing a synchronized method for an object, all other threads that invoke synchronized methods for the same object suspend execution until the first thread is done with the object;
- When a synchronized method exits, it automatically establishes a happens-before relationship with any subsequent invocation of a synchronized method for the same object. This guarantees that changes to the state of the object are visible to all threads

151

<https://docs.oracle.com/javase/tutorial/essential/concurrency/sync.html>